

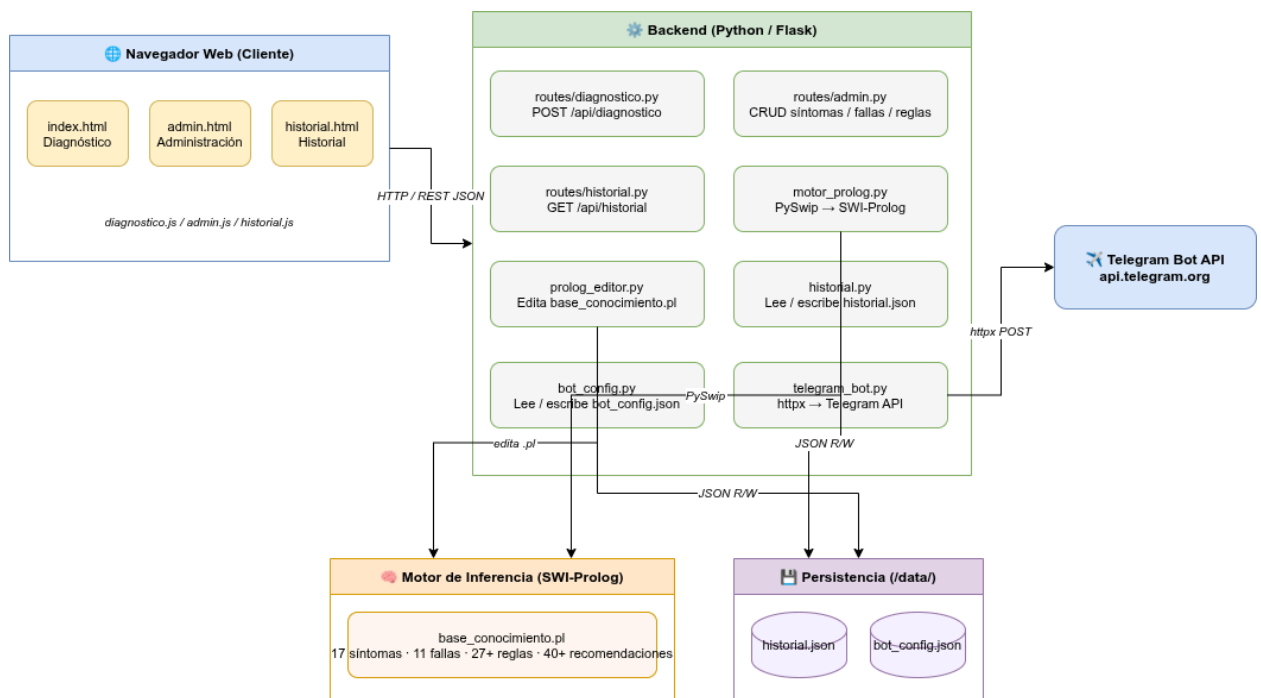
Manual Técnico — Doctor Byte

1. Descripción general

Doctor Byte es un sistema experto para el diagnóstico de fallas comunes en computadoras. El usuario selecciona síntomas desde una interfaz web y el sistema aplica reglas de inferencia implementadas en Prolog para identificar la falla más probable y emitir recomendaciones. Cada diagnóstico genera una notificación automática a través de un bot de Telegram.

2. Arquitectura del sistema

Diagrama de capas



Flujo de un diagnóstico

- El usuario selecciona síntomas en `index.html`
- El frontend envía `POST /api/diagnostico` con la lista de síntomas
- `motor_prolog.py` consulta a SWI-Prolog mediante PySwip
- Prolog ejecuta `diagnostico_completo/3` y devuelve falla + recomendaciones
- El backend guarda el resultado en `historial.json`
- `telegram_bot.py` envía la notificación al chat configurado
- El frontend muestra el diagnóstico al usuario

3. Tecnologías utilizadas

Capa	Tecnología	Versión
Motor de inferencia	SWI-Prolog + PySwip	9.x
Backend	Python + Flask	3.11 / 3.x
Frontend	HTML5 + CSS3 + JavaScript	—
Estilos	Bootstrap 5.3 + Bootstrap Icons	5.3.3
Notificaciones	Telegram Bot API (httpx)	—
Contenedores	Docker + Docker Compose	—
Persistencia	JSON (sin base de datos adicional)	—

4. Estructura del proyecto

```
Dr_Byte_FASE1/
├── backend/
│   ├── app.py                # Punto de entrada Flask
│   ├── config.py             # Variables de configuración
│   ├── motor_prolog.py       # Interfaz con SWI-Prolog (PySwip)
│   ├── prolog_editor.py      # Lectura/escritura de base_conocimiento.pl
│   ├── historial.py          # Gestión del historial JSON
│   ├── bot_config.py         # Configuración persistente del bot
│   ├── telegram_bot.py       # Envío de notificaciones
│   └── routes/
│       ├── diagnostico.py    # POST /api/diagnostico
│       ├── admin.py          # CRUD síntomas, fallas, reglas
│       └── historial.py      # GET /api/historial
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/
│   ├── index.html            # Pantalla de diagnóstico
│   ├── admin.html            # Panel de administración
│   ├── historial.html        # Vista de historial
│   ├── css/styles.css
│   ├── js/
│   │   ├── diagnostico.js
│   │   ├── admin.js
│   │   └── historial.js
│   ├── nginx.conf
│   └── Dockerfile
├── prolog/
│   └── base_conocimiento.pl  # Base de conocimiento
└── data/
```

```
|   |─ historial.json
|   └─ bot_config.json
└─ docs/
    |─ manual_tecnico.md
    └─ manual_usuario.md
└─ docker-compose.yml
└─ .env.example
└─ README.md
```

5. Requisitos previos

- Docker y Docker Compose instalados
- Cuenta de Telegram con un bot creado desde @BotFather
- Token del bot y Chat ID de destino

6. Configuración inicial

6.1 Variables de entorno

Copiar el archivo de ejemplo y completar los valores:

```
cp .env.example .env
```

Contenido de .env:

```
TELEGRAM_TOKEN=<token_obtenido_de_botfather>
TELEGRAM_CHAT_ID=<id_del_chat_destino>
```

Para obtener el TELEGRAM_CHAT_ID del usuario o grupo destino, iniciar una conversación con el bot y consultar:

```
https://api.telegram.org/bot<TOKEN>/getUpdates
```

6.2 Ejecución

```
docker-compose up --build
```

Servicio	URL
Frontend	http://localhost:8080
Backend API	http://localhost:5000

7. Base de conocimiento Prolog

El archivo prolog/base_conocimiento.pl contiene:

Predicado	Descripción	Cantidad
sintoma/1	Declara un síntoma válido	17
descripcion_sintoma/2	Texto legible del síntoma	17
falla/1	Declara una falla diagnosticable	11
descripcion_falla/2	Texto legible de la falla	11
sintoma_falla/2	Regla de inferencia síntoma → falla	27+
recomendacion/2	Recomendaciones por falla	40+

Predicados del motor de inferencia

```
% Cuenta coincidencias entre síntomas seleccionados y una falla
puntaje_falla(Sintomas, Falla, Puntaje)

% Selecciona la falla con mayor puntaje (usa corte para única solución)
mejor_diagnostico(Sintomas, Falla) :- ..., !.

% Punto de entrada principal
diagnostico_completo(Sintomas, Falla, Recomendaciones)
```

El motor utiliza un algoritmo de conteo de coincidencias: para cada falla candidata cuenta cuántos síntomas seleccionados la tienen como destino en sintoma_falla/2. La falla con mayor conteo gana. El corte (!) garantiza una única solución.

Uso de listas en Prolog

```
% findall construye listas de coincidencias
findall(1, (member(S, Sintomas), sintoma_falla(S, Falla)), Coincidencias)

% member/2 verifica pertenencia a lista
member(S, ListaSintomas)

% Recursión con cabeza/cola de lista
contar_coincidencias([S|Resto], ListaSintomas, Total)
```

8. API REST — Endpoints principales

Diagnóstico

Método	Ruta	Descripción
GET	/api/sintomas	Lista todos los síntomas disponibles

POST	/api/ diagnostico	Ejecuta el diagnóstico con los síntomas enviados
------	-------------------	--

Body de /api/ diagnostico:

```
{ "síntomas": ["pantalla_negra", "sin_imagen", "reinicio_inesperado"] }
```

Respuesta:

```
{ "id": "D-00042", "fecha": "2026-06-13 10:45:00",
  "síntomas": ["pantalla_negra", "sin_imagen", "reinicio_inesperado"],
  "falla": "falla_gpu", "descripcion": "Problema con la tarjeta gráfica",
  "recomendaciones": ["Verifica el cable de video...", "..."] }
```

Administración

Método	Ruta	Descripción
GET	/api/admin/síntomas	Listar síntomas
POST	/api/admin/síntomas	Crear síntoma
PUT	/api/admin/síntomas/<id>	Editar descripción
DELETE	/api/admin/síntomas/<id>	Eliminar síntoma y sus reglas
GET	/api/admin/fallas	Listar fallas
POST	/api/admin/fallas	Crear falla
PUT	/api/admin/fallas/<id>	Editar descripción
DELETE	/api/admin/fallas/<id>	Eliminar falla, reglas y recomendaciones
GET	/api/admin/reglas	Listar reglas sintoma→falla
POST	/api/admin/reglas	Crear regla
PUT	/api/admin/reglas/<sint>/<falla>	Editar falla destino de la regla
DELETE	/api/admin/reglas/<sint>/<falla>	Eliminar regla
GET	/api/admin/recomendaciones	Listar recomendaciones
POST	/api/admin/recomendaciones	Agregar recomendación
PUT	/api/admin/recomendaciones	Editar texto de recomendación
DELETE	/api/admin/recomendaciones	Eliminar recomendación
GET	/api/admin/config/bot	Obtener configuración del bot
PUT	/api/admin/config/bot	Actualizar configuración del bot

Historial

Método	Ruta	Descripción
GET	/api/historial	Lista todos los diagnósticos anteriores

9. Persistencia de datos

El sistema utiliza dos archivos JSON dentro del volumen /data/:

historial.json — se actualiza con cada diagnóstico

```
[{ "id": "D-00001", "fecha": "2026-06-10 14:32:00",
  "sintomas": ["pantalla_negra", "no_enciende"],
  "falla": "falla_fuente_poder",
  "descripcion": "Problema con la fuente de alimentación",
  "recomendaciones": ["..."] }]
```

bot_config.json — configuración activa del bot

```
{ "token": "", "chat_id": "", "activo": true,
  "encabezado": "Nuevo diagnóstico - Doctor Byte" }
```

Cuando los campos token o chat_id están vacíos, el sistema usa los valores del archivo .env como respaldo.

10. Configuración del Bot de Telegram

El bot se configura desde el panel de administración (/admin.html) sin necesidad de reiniciar el contenedor. Los cambios surten efecto en el siguiente diagnóstico.

Campo	Descripción
Token	Credencial del bot obtenida de @BotFather
Chat ID	Destino de las notificaciones (usuario, grupo o canal)
Activo	Si está desactivado, envía mensaje de mantenimiento
Encabezado	Primera línea personalizable del mensaje

Nota de seguridad: el token y el chat ID nunca están escritos en el código fuente. Se leen desde variables de entorno (.env) y pueden sobrescribirse desde el panel de administración (bot_config.json).

11. Patrones de arquitectura utilizados

Patrón	Aplicación
Blueprint (Flask)	Separación de rutas en módulos independientes
Motor de inferencia	Prolog como motor, Python como orquestador
Fallback en configuración	.env como valor por defecto, JSON como configuración activa
Lock de concurrencia	threading.Lock() en accesos al archivo Prolog y bot_config.json
Bind mount Docker	./data:/data para persistencia visible en el sistema de archivos del host